

1 Introduction

This is the report for the first assignment in the Computer Systems course at DIKU, copenhagen university. In this assignment we have been tasked with the job of making a file(1) program, which can tell the user which format the file is in. Through out this report we will give an overview of the design and implementation of our program as well as evaluate our results

2 Analysis

The file command is a unix command with the ability to tell the type of a file, such as ASCII or empty.

The following is an example of the functionality of file:

```
$ file testfile
testfile: ASCII text
```

We wish to implement the file command by using C. Our program is called `./file` to differentiate the two. We have set the following requirements for our implementation:

- Empty: The `./file` must recognize an empty file as a file with no bytes in it.
- ASCII: `./file` must be able to tell ASCII data, of ascii characters 7-13, 27, 32-126
- ISO-8859: It must also be able to tell latin-1 and other ISO-8859 encodings
- UTF-8 unicode: It should also be able to tell UTF-8 files.
- Data: if no other data type applies, or if the testfile consists of multiple, `./file` must report data as the type.

Our `./file` implementation should not recognize the difference between a testfile ending in a new line and testfile ending without line termination.

3 Design

We will in this part of the assignment talk about some of the design choices we have made and our thoughts leading up to the implementation. We wanted to have an as easy as possible main function, so for that we decided to implement helper functions for each of the file types we were tasked to find. And then checking them one by one, if none of them were matched we would return the data type. By doing it this way, we were able to keep the main function quite simple, which furthermore allowed us to have a good view of the implementation of the API. All of our functions are created in a way so that the second a given character isn't part of the data structure we are searching through it stops. That way we don't spend unnecessary time searching through a file that we already know isn't a part of the set we are looking for. In general, having

something that works has been the most important part of our design, and the speed in which it runs and the way it's built have been a second priority. In general, we have just tried to keep it simple and then refactored the places we saw fit.

4 Implementation

Our implementation is built around our helper functions so that they each have the job of checking for one file type. All of the helper functions are of the type `int`, by doing this we can have our function return 1 in case the file upholds the given requirements for a file type or return 0 in case the read file isn't a part of a given data structure. That's also why it's important that in our main function, run the helper functions in the correct order. This is because the ASCII -file type is a subset of the ISO-8859-1 file type and that is a subset of the UFT8 file type. So let's say we first check for ISO and then ASCII we would never get an ASCII return but always an ISO, which would be wrong. So the entirety of the system relies on the fact that the previous file structures have been checked. For checking the files we use the command "`fgetc`", it's useful as this has the ability to check one character at a time and then we check if that character adheres to the file specifications. An example of this could be the ISO implementation:

```
int is_ISO(char *file_path) {  
    FILE *file = fopen(file_path, "rb");  
    int c;  
    while ((c = fgetc(file)) != EOF) {  
        if (!((c > 6 && c < 14) || c == 27 || (c > 31 && c < 127) || (c > 159 && c <= 255))) {  
            fclose(file);  
            return 0;  
        }  
    }  
    fclose(file);  
    return 1;  
}
```

In the above code snippet, we use the abilities from "`fgetc`", where it reads one character at the time, and if the given character isn't in the specified intervals it returns zero. This way we check each character against the requirements from the file type until the "End Of File (EOF)", and if all characters match it returns a 1. The other important thing is that in case we come to a solution whether true or false, we remember to close the file, so we don't have multiple files open at the same time.

A not-so-trivial part of our implementation is the UTF8 checker. Here our goal is to check if all the characters in a file is UFT8. In doing so we need to first make sure how large the first byte is, because that is an indicator of how many more bytes come afterward. We converted the binary numbers we us to decimals because it was easier for us to understand what was going on. We have a while loop which checks if we reach the end of the file, and then we inside that have a series of if statements that check the number of 1 we have leading the byte to know how many following bytes we need to check. Let's

look at an example:

```
else if ((c & 224) == 192)
{
    c = fgetc(file);
    if ((c & 192) != 128) {
        fclose(file);
        return 0;
    }
}
```

Here is the start of the checks. We first check that the leading byte which we assigned earlier to `c`, is equal to the binary pattern of a two-byte sequence, so we check if it starts with two ones followed by a 0. If that's the case, we then set a new `c`, by using the `fgetc` function, and we check if that is equal to the requirements from the assignment, which is to check if it starts with a 1 and is followed by a zero. If that is not the case, it's not a UTF8 sequence and we return zero. It's the same thing we do for 3 and 4-byte sequences. Here we just have a for loop which checks that each of the following bytes follows the 10.. pattern that was given in the assignment. We tried making the main function as simple and well-written as possible. Which has given us a good overview of our implementation. We have implemented everything from the API except the ability to read secret files. But we return the requested response in all other cases. The lack of this was mainly due to us not being able to create secret files, and therefore not being able to test our implementation, in which case we decided it would be better not to implement instead of trying on something we couldn't test.

5 Testing

Testing is done by using a bash script which prints files including different data types: ASCII, ISO-8859, UTF-8, data, and empty files. Files consist of hexadecimal representation of numbers using:

Test did cover every Condition of statements, by generating files with hexadecimal format. Some tests were not able to pass like some ISO test. UTF recognition is covered for all conditions and works as intended. Test for some functions includes both sequences and single inputs. Some few tests for invalid input were also tested, and results as data-type.

6 Conclusion

We have in this assignment worked with file types, and tried to understand when a file is a certain type. We have implemented the requirements to the best of our ability and believe that they should work as intended. We have tried to make tests that cover as much as possible. We have some problems in understanding where it passes and does not, as the test script doesn't always follow the assignment-given specifications. But we believe that our tests return what we expect them to.